

From Vibe Coding to Spec-Driven Agentic Engineering

A Hands-On Implementation using Claude Code

PART XV

Instructor: Muhammad Arif Butt, PhD

About The Instructor



Dr. Muhammad Arif Butt
Asst. Prof. Department of Data Science
University of Punjab, Lahore
arif@pucit.edu.pk
https://arif.excaliat@gmail.com
https://arifbutt.me
https://www.linkedin.com/in/dr-arif-butt/
https://youtube.com/learnwitharif
https://github.com/arifpucit
https://huggingface.co/arif-butt

- **Education:**
  - Graduation from Pakistan Military Academy, Kakul
  - MPhil CS from University of Punjab, Lahore
  - PhD CS from University of Punjab, Lahore
- **Experience:**
  - Served in field/staff/instructional posts in Pakistan Army
  - Assistant Professor, Department of Data Science
  - Founder Excaliat (https://excaliat.com/en)
  - Founder FalconHunt (https://falconhunt.org/)
  - Co-Founder Tbox Solutionz (https://tboxsolutionz.com/)
- **Research Interest:**
  - Embedded and Real-Time Operating Systems
  - Vulnerability Analysis, Binary Exploitation & Exploit Development
  - AI-Driven Cybersecurity and Securing AI systems

Scheme of Sessions

● Today's session — Part XV

Session	Topic
Part I	Overview of Agentic Coding Assistants
Part II	Claude Ecosystem, Installing & Using Claude Code
Part III	Slash Commands, Context Window & .claude Directory
Part IV	Workflow of Building a Basic Vulnerable Web Application
Part V	Exploiting Vulnerable Web Application
Part VI	Workflow of Implementing Features and Fixing Bugs
Part VII	Collaborative Workflow: Claude Code + GitHub + Jira
Part VIII	Built-in Skills & Vuln-Web-App Current Status
Part IX	Creating & Using Custom Skills & Skill Marketplace

Session	Topic
Part X	Deep Dive: Conceptual Understanding of MCP
Part XI	MCP in Action
Part XII	Creating / Using Built-in & Custom Subagents
Part XIII	Git Worktrees, Agent Teams & a Revisit to OWASP Top 10
Part XIV	A2A Protocol with a Hands-on Demonstration
Part XV	**Permissions in Claude Code**
Part XVI	Sandboxing in Claude Code
Part XVII	Hooks and Guardrails in Claude Code

Permissions in Claude Code

Instructor: Muhammad Arif Butt, PhD

AI Coding Agents Needs Safety

Why Permissions & Sandboxing — Capability Needs Boundaries

Skills, MCP servers and subagents gave Claude real reach on a real machine. Two independent mechanisms decide what it may do — and where it may do it.

WE KEPT ADDING CAPABILITY

- **Skills** — packaged know-how it can load
- **MCP servers** — outside tools and data
- **Subagents** — parallel workers of its own
- **Bash** — runs real commands
- **Edit / Write** — changes your files
- **WebFetch** — pulls in outside content

Every one of them ends in a real command, running as you.

WHY GOOD INTENTIONS AREN'T A BOUNDARY

- **Honest mistake** — The right command in the wrong directory — or on the wrong branch.
- **Prompt injection** — A web page, issue or README quietly tells Claude what to do next.
- **Blast radius** — Secrets, sibling repos and the open network are one command away.

You can't review your way out of this — reviewing every single call is the thing you were trying to stop doing.

TWO INDEPENDENT ANSWERS

PERMISSIONS — up next

WHAT Claude may do

ENFORCED BY
Claude Code, before the call runs

COVERS
every tool — Bash, Read, Edit, WebFetch, MCP

JUDGES
the command it is about to run

A BLOCK LOOKS LIKE
the call never starts

STILL HOLDS WHEN
Claude decides badly

SANDBOX — after that

WHERE Claude may do it

ENFORCED BY
the operating system, while it runs

COVERS
Bash commands and their child processes

JUDGES
filesystem and network boundaries

A BLOCK LOOKS LIKE
the syscall is simply refused

STILL HOLDS WHEN
Claude is fooled — or a command does more than its name suggests

Permissions can be argued with by a clever prompt. A kernel boundary cannot.

DEFENSE IN DEPTH: `CLAUDE.md` asks nicely, enforces nothing › `Permissions` which tool calls are allowed, before they run › `Sandbox` which files and hosts Bash may reach, OS-enforced › `Hooks` your own script gets the final say

Guidance is not enforcement.
Your prompt and `CLAUDE.md` shape what Claude tries; permissions and the sandbox decide what actually happens. And the boundary pays for itself — inside the sandbox, commands can run without stopping to ask.

Claude Control Center: `settings.json`

settings.json — The Control Center

One declarative JSON file that configures Claude Code — resolved through one precedence chain, and hot-reloaded the moment you save it.

PRECEDENCE — HIGHEST SOURCE WINS

WINS

Managed `managed-settings.json` — locked · cannot be overridden
System-wide policy set by your organization.

CLI flags `--settings, --model` — this session only
Flags you pass when launching Claude Code.

Local `.claude/settings.local.json` — yours · gitignored
Your private tweaks for this project, uncommitted.

Project `.claude/settings.json` — team · committed
Shared with everyone working in this repo.

User `~/.claude/settings.json` — your global defaults
Your baseline for every project on this machine.

~/.claude/settings.json

```jsonc
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "model": "claude-sonnet-5",
  "fallbackModel": ["claude-opus-4.6", "haiku"],
  "outputStyle": "Default",                // "Proactive" | "Explanatory" | "Learning"
  "defaultShell": "bash",                  // "powershell"
  "cleanupPeriodDays": 30,
  "env": { "NODE_ENV": "development", "MY_API_BASE_URL": "https://api.example.com" },
  "statusLine": { "type": "command", "command": "~/.claude/statusline.sh", "padding": 2 },

  // covered on the next slides
  "permissions": { },
  "sandbox": { },
  "hooks": { }
}
```

Configure it, don't hand-edit it.
`/config` writes settings back to the right scope for you. Most keys reload live — only `model` and `outputStyle` wait for a restart.

Permissions: Your First Line of Defense

Permissions — Your First Line of Defense

Fine-grained rules and modes that decide what Claude may do without stopping to ask — enforced by Claude Code itself, not by the model.

EVERY TOOL CALL RUNS THIS GAUNTLET

Tool call (Claude wants to act) › **deny** (hard block) › **ask** (always prompt) › **allow** (auto-approve) › **mode** (no rule matched)

First match wins — a deny can't carry allow-list exceptions, and an ask outranks a narrower allow.

SIX MODES SET THE BASELINE

⇧Tab cycles the top row · auto, dontAsk and bypassPermissions are opt-in (flag or settings)

`plan` read-only	`default` manual · prompts	`acceptEdits` edits in cwd
`auto` classifier checks	`dontAsk` prompt → denial	`bypassPermissions` explicit asks still prompt

most oversight ... fewest prompts

INSIDE THE PERMISSIONS BLOCK

1. **Mode** the fallback when no rule matches — set with `defaultMode`
2. **Allow** runs with no prompt — `Tool(specifier)`, e.g. Bash(uv run *)
3. **Ask** forces a prompt even when a broader allow rule also matches
4. **Deny** blocks in every mode — a bare `Bash` removes the tool from context
5. **Reach** widens file access with `additionalDirectories` — nothing more

`.claude/settings.json`

```
{
  "permissions": {
    "defaultMode": "default",       // acceptEdits · plan · auto · dontAsk · bypassPermissions

    "allow": [                      // runs with no prompt
      "Read(**)", "Edit(app/**)", "Edit(pyproject.toml)",
      "Bash(python *)", "Bash(uv run *)", "Bash(uv sync)",
      "Bash(git status)", "Bash(git diff *)", "Bash(git log *)"
    ],
    "ask": [                        // always prompt
      "Read(.env*)", "Edit(.env*)", "Bash(uv add *)",
      "Bash(git add *)", "Bash(git commit *)", "Bash(git push *)"
    ],
    "deny": [                       // hard block · wins everywhere
      "Bash(rm -rf *)", "Bash(sudo *)", "Bash(psql *)",
      "Bash(git reset --hard *)", "Read(secrets/**)", "Read(.env.local)"
    ],
    "additionalDirectories": ["../docs/", "../shared/"]
  }
}
```

Deny beats everything.

A deny rule holds even in `bypassPermissions`, and a user-level deny beats a project-level allow. But deny only covers Claude's own file and Bash tools — for a real boundary add `PreToolUse` hook or enable the sandbox.

Hands on Practice: Permissions



https://github.com/arifpucit/cc-perms-sandbox

Q/A



Thank you!